

INFORME TÉCNICO DE EVALUACIÓN

Sistema de Gestión Escolar Colegio Bernardo O'Higgins

DSY1106 — Desarrollo Fullstack III

Evaluación Parcial N°3: Integración de Arquitectura de Microservicios Junio 2026

Aspecto	Detalle
Patrón Arquitectónico	Microservicios (4 servicios backend + 1 frontend)
Framework Backend	Java 21 + Spring Boot 4.0.6 + Spring Cloud 2025.1.1
Framework Frontend	React 18 + Vite + TailwindCSS
Bases de Datos	MySQL 8.0 (2 instancias) + MongoDB 6.0
Mensajería	RabbitMQ 3-management (Pub/Sub)
Infraestructura	Docker Compose (10 contenedores) + Nginx
Calidad de Código	SonarQube Community + JaCoCo + Swagger/OpenAPI

Índice

Índice	2
1. Arquitectura General	4
1.1 Diagrama de Componentes	4
1.2 Flujo de Datos Principal — Anotación Negativa	4
2. Frontend (React + Vite + TailwindCSS)	5
2.1 Stack Tecnológico	5
2.2 Componentes Personalizados	6
2.3 Páginas del Sistema (20+ páginas)	6
2.3.1 Módulo UTP — Administración	6
2.3.2 Módulo Profesor	6
2.4 useState() y useEffect() — Casos de Uso	7
2.5 Context API — Estado Global	7
2.6 Capa API (Axios)	7
3. Backend — Microservicios Spring Boot	8
3.1 API Gateway (puerto 9090)	8
3.1.1 Enrutamiento (application.properties)	8
3.1.2 Seguridad — Spring Security + JWT	8
3.1.3 Circuit Breaker Resilience4j — API Gateway	8
3.2 Servicio Académico (puerto 8081)	8
3.2.1 Controladores REST	8
3.2.2 Patrón Service (Interface + Implementation)	9
3.3 Servicio Asistencia (puerto 8082)	9
3.3.1 OpenFeign — Cliente HTTP Declarativo	9
3.3.2 RabbitMQ — Publicador de Eventos	9
3.4 Servicio Comunicaciones (puerto 8083)	10
3.4.1 RabbitMQ — Consumidor de Eventos	10
3.4.2 Modelos MongoDB	10
3.5 Swagger / OpenAPI	10
4. Persistencia de Datos	11
4.1 Estrategia por Servicio	11
4.2 Spring Data JPA — Servicios Académico y Asistencia	11
4.3 Spring Data MongoDB — Servicio Comunicaciones	11
4.4 Persistencia con Docker Volumes	11
5. Pruebas Unitarias — JUnit 5	12
5.1 Herramientas de Testing	12
5.2 Tests por Servicio	12
5.2.1 API Gateway	12
5.2.2 Servicio Académico	12
5.2.3 Servicio Asistencia	12
5.2.4 Servicio Comunicaciones	13
5.3 Configuración JaCoCo + SonarQube	13
5.4 Resultados SonarQube — Cobertura Real	13

6. Infraestructura — Docker	15
6.1 Docker Compose (docker-compose.yml)	15
6.2 Healthchecks Configurados	15
6.3 Dockerfiles	15
7. Patrones de Diseño	16
7.1 Patrones Implementados	16
8. Justificación de la Arquitectura de Microservicios	16
8.1 Por qué Microservicios para el Sistema Escolar	16
8.2 Cumplimiento de Requerimientos del Cliente	16
8.3 Recomendaciones de Mejora Futura	17

1. Arquitectura General

El sistema implementa un patrón de arquitectura de microservicios compuesto por cuatro servicios backend independientes y un frontend React SPA (Single Page Application). Cada servicio posee su propia base de datos, garantizando el aislamiento de datos y la escalabilidad horizontal independiente.

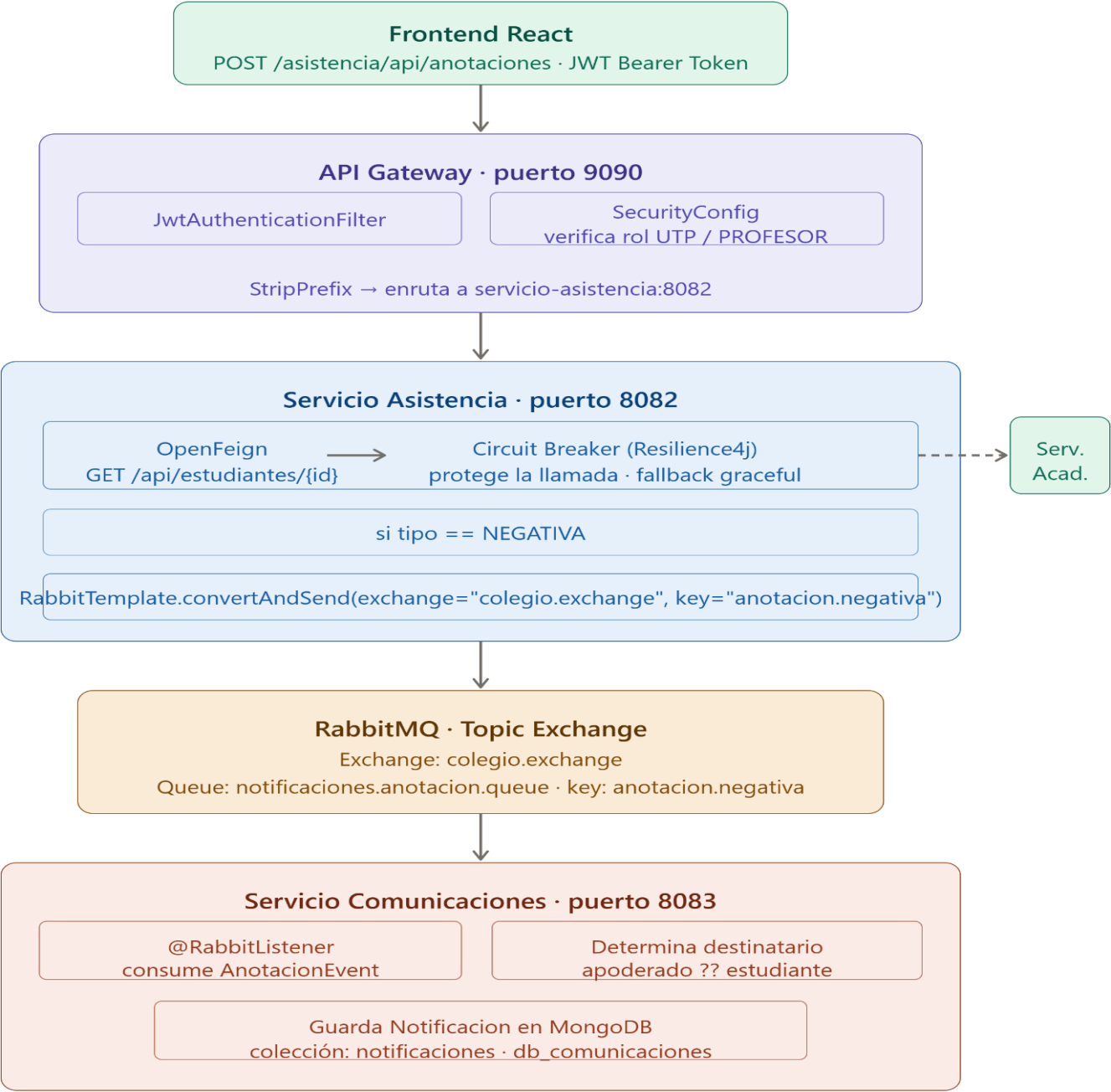
1.1 Diagrama de Componentes

La arquitectura se organiza en las siguientes capas:

Capa	Componente	Tecnología	Puerto
Presentación	Frontend React (Nginx)	React 18 + Vite + TailwindCSS	3000 → 80
Punto de Entrada	API Gateway	Spring Cloud Gateway + Spring Security + JWT	9090
Negocio	Servicio Académico	Spring Boot WebMVC + JPA + MySQL	8081
Negocio	Servicio Asistencia	Spring Boot + JPA + OpenFeign + RabbitMQ	8082
Negocio	Servicio Comunicaciones	Spring Boot + MongoDB + RabbitMQ Consumer	8083
Mensajería	RabbitMQ	rabbitmq:3-management	5672 / 15672
Calidad	SonarQube	sonarqube:community	9000

1.2 Flujo de Datos Principal — Anotación Negativa

El flujo más representativo del sistema integra todos los microservicios a través de la publicación y consumo de eventos asíncronos:



2. Frontend (React + Vite + TailwindCSS)

2.1 Stack Tecnológico

Archivo de configuración: frontend/package.json

Tecnología	Versión	Archivo de Configuración
React + JSX	^18.3.1	frontend/package.json
Vite (bundler)	^5.3.1	frontend/vite.config.js
TailwindCSS	^3.4.4	frontend/tailwind.config.js, frontend/postcss.config.js

React Router DOM v6	^6.23.1	frontend/package.json
Axios (HTTP Client)	^1.7.2	frontend/src/api/client.js
Lucide React (Iconos)	^0.378.0	frontend/package.json
Recharts (Gráficos)	^2.12.7	frontend/package.json
html2canvas + jsPDF	^1.4.1 / ^4.2.1	frontend/package.json (reportes PDF)
TypeScript (dev)	^5.4.5	frontend/package.json

2.2 Componentes Personalizados

Ubicación: frontend/src/components/

Componente	Archivo	Función Principal
Layout	Layout.jsx	Sidebar responsive, Dark Mode, header con menú usuario, badges de notificaciones (polling cada 30s)
ProtectedRoute	ProtectedRoute.jsx	Guard de autenticación: verifica token y rol; redirige a /login o /dashboard según permisos
Skeleton	Skeleton.jsx	Componente compuesto con variantes: Text, Circle, Rectangle, Dashboard, Table (estados de carga)
Pagination	Pagination.jsx	Paginación reutilizable: números, botones primero/último, puntos suspensivos, indicador total

2.3 Páginas del Sistema (20+ páginas)

2.3.1 Módulo UTP — Administración

Página	Archivo	Funcionalidad
Dashboard	DashboardHome.jsx	Estadísticas adaptativas por rol
Login	Login.jsx	Autenticación + acceso rápido por rol
Diagnóstico	Diagnostico.jsx	Verificación conectividad servicios
Usuarios UTP	UtpUserList.jsx + UtpUserForm.jsx	CRUD administradores UTP
Estudiantes	EstudianteList.jsx + EstudianteForm.jsx	CRUD estudiantes
Profesores	ProfesorList.jsx + ProfesorForm.jsx	CRUD profesores
Apoderados	ApoderadoList.jsx + ApoderadoForm.jsx	CRUD apoderados
Asignaturas	AsignaturaList.jsx + AsignaturaForm.jsx	CRUD asignaturas
Evaluaciones	EvaluacionList.jsx + EvaluacionForm.jsx	CRUD evaluaciones

2.3.2 Módulo Profesor

Página	Archivo	Funcionalidad
Notas	notas/NotasList.jsx	Registro y gestión de calificaciones

Asistencia	asistencia/AsistenciaList.jsx	Toma y gestión de asistencia (PRESENTE/ATRASO/AUSENTE/JUSTIFICADO)
Anotaciones	anotaciones/AnotacionList.jsx	Registro de anotaciones positivas/negativas (activa RabbitMQ)
Horarios	horarios/HorariosPage.jsx	Visualización de horarios de clases
Reportes	reportes/ReportesPage.jsx	Estadísticas y exportación a PDF
Notificaciones	notificaciones/NotificacionesPage.jsx	Centro de notificaciones con badges en tiempo real

2.4 useState() y useEffect() — Casos de Uso

Ambos hooks se utilizan de forma extensiva en TODAS las páginas y componentes del frontend:

Archivo	useState	useEffect
Layout.jsx	sidebarOpen, userMenuOpen, badges, dark	Polling 30s: conteo mensajes/notificaciones no leídos; persistencia dark mode localStorage
Login.jsx	rut, password, error, loading, showPassword	— (event-driven en submit asíncrono)
DashboardHome.jsx	loading, stats (por rol)	Promise.allSettled: 9 llamadas API simultáneas con estadísticas por rol
AuthContext.jsx	user (inicializado desde localStorage)	Verificación de token JWT al montar el componente
Diagnostico.jsx	results, testing	Diagnóstico secuencial de conectividad de servicios
EstudianteList.jsx	estudiantes, loading, search, page	Carga paginada de estudiantes; refetch en cambio de búsqueda/página

2.5 Context API — Estado Global

Contexto	Archivo	Provee
AuthContext	frontend/src/context/AuthContext.jsx	user, login(), logout(), isAuthenticated, hasRole() — persistencia en localStorage
DarkModeContext	frontend/src/components/Layout.jsx	dark, toggle() — respeta prefers-color-scheme del sistema; persiste en localStorage

2.6 Capa API (Axios)

- frontend/src/api/client.js — Interceptor de peticiones: agrega Authorization: Bearer {token}; interceptor de respuestas: 401 → limpia sesión → redirige a /login
- frontend/src/api/auth.js — login(data), cambiarContraseña(data), resetearContraseña(data)
- frontend/src/api/endpoints.js — APIs agrupadas por dominio: estudiantesApi, profesoresApi, asistenciasApi, anotacionesApi, mensajesApi, notificacionesApi, horariosApi, usuariosUtpApi

3. Backend — Microservicios Spring Boot

3.1 API Gateway (puerto 9090)

Archivo clave: api-gateway/pom.xml | Spring Boot 4.0.6 + Spring Cloud 2025.1.1

3.1.1 Enrutamiento (application.properties)

Ruta	Destino	Filtro
/academico/**	http://servicio-academico:8081	StripPrefix=1
/asistencia/**	http://servicio-asistencia:8082	StripPrefix=1
/comunicaciones/**	http://servicio-comunicaciones:8083	StripPrefix=1

3.1.2 Seguridad — Spring Security + JWT

Archivo: api-gateway/src/main/java/com/colegio/api_gateway/security/

- SecurityConfig.java — SecurityFilterChain con CORS, CSRF disabled, reglas de autorización por rol y endpoint
- JwtUtil.java — Generación/validación JWT con jjwt 0.12.6, HMAC-SHA, claims: subject (RUT) + role, expiración 8 horas
- AuthService.java — Autenticación UTP (local, BCrypt en ConcurrentHashMap) y remota (RestTemplate + CircuitBreaker)
- JwtAuthenticationFilter.java — Filtro que extrae y valida el Bearer Token en cada petición

3.1.3 Circuit Breaker Resilience4j — API Gateway

Parámetro	Valor
Instancia	auth-academico
sliding-window-size	10
failure-rate-threshold	50%
wait-duration-in-open-state	15 segundos
permitted-calls-in-half-open	3
TimeLimiter timeout	5 segundos
Fallback behavior	Retorna Optional.empty() → login falla gracefully

3.2 Servicio Académico (puerto 8081)

Archivo clave: servicio-academico/pom.xml | Base de datos: MySQL 8.0 (db_academico) | ORM: Spring Data JPA (Hibernate)

3.2.1 Controladores REST

Controlador	Endpoints Principales
ApoderadoController	CRUD apoderados, búsqueda por RUT, búsqueda por estudiante
EstudianteController	CRUD estudiantes, búsqueda por RUT, asociación con cursos
ProfesorController	CRUD profesores, autenticación remota
AsignaturaController	CRUD asignaturas, filtrado por nivel de curso

EvaluacionController	CRUD evaluaciones, filtrado por tipo (TipoEvaluacion)
NotaController	CRUD notas, búsqueda por estudiante y evaluación
HorarioController	CRUD horarios, búsqueda por curso/día/profesor
AuthController	Validación de credenciales (consumido por API Gateway vía RestTemplate)

3.2.2 Patrón Service (Interface + Implementation)

Ubicación: servicio-academico/src/main/java/com/colegio/servicio_academico/service/

Interfaz	Implementación
ApoderadoService	impl/ApoderadoServiceImpl
EstudianteService	impl/EstudianteServiceImpl
ProfesorService	impl/ProfesorServiceImpl
NotaService	impl/NotaServiceImpl
EvaluacionService	impl/EvaluacionServiceImpl
HorarioService	impl/HorarioServiceImpl
AsignaturaService	impl/AsignaturaServiceImpl

3.3 Servicio Asistencia (puerto 8082)

Archivo clave: servicio-asistencia/pom.xml | Base de datos: MySQL 8.0 (db_asistencia) | Comunicación: OpenFeign + Circuit Breaker | Mensajería: RabbitMQ (Publisher)

3.3.1 OpenFeign — Cliente HTTP Declarativo

Archivo: servicio-asistencia/src/main/java/com/colegio/servicio_asistencia/feign/

```
@FeignClient(name = "servicio-academico",
            url = "${academico.service.url}",
            fallback = AcademicoFeignClientFallback.class)
public interface AcademicoFeignClient {
    @GetMapping("/api/estudiantes/{id}")
    Map<String, Object> buscarEstudiantePorId(@PathVariable Long id);
    @GetMapping("/api/apoderados/por-estudiante/{id}")
    List<Map<String, Object>> buscarApoderadosPorEstudiante(@PathVariable Long id);
}
```

Fallback (AcademicoFeignClientFallback.java):

- buscarEstudiantePorId → retorna null + log WARNING
- buscarApoderadosPorEstudiante → retorna Collections.emptyList() + log WARNING

3.3.2 RabbitMQ — Publicador de Eventos

Ubicación: servicio-asistencia/src/main/java/com/colegio/servicio_asistencia/messaging/

Archivo	Elemento	Valor
RabbitMQConfig.java	Exchange	colegio.exchange (Topic Exchange)
RabbitMQConfig.java	Queue	notificaciones.anotacion.queue
RabbitMQConfig.java	Routing Key	anotacion.negativa

RabbitMQConfig.java	Serialización	Jackson.JsonMessageConverter (JSON)
AnotacionEventPublisher.java	Método	publicar(AnotacionEvent) → rabbitTemplate.convertAndSend()
AnotacionEvent.java	DTO	estudianteld, apoderadold, descripcion, fecha

3.4 Servicio Comunicaciones (puerto 8083)

Archivo clave: servicio-comunicaciones/pom.xml | Base de datos: MongoDB 6.0 (db_comunicaciones) | Mensajería: RabbitMQ (Consumer)

3.4.1 RabbitMQ — Consumidor de Eventos

Archivo: AnotacionEventConsumer.java

```
@RabbitListener(queues = "notificaciones.anotacion.queue")
public void procesarAnotacionNegativa(AnotacionEvent evento) {
    String destinatarioId = (evento.getApoderadoId() != null)
        ? evento.getApoderadoId().toString()
        : evento.getEstudianteId().toString();
    // Crea y guarda Notificacion en MongoDB
}
```

3.4.2 Modelos MongoDB

Modelo	Atributos Clave
Mensaje	id (ObjectId), remitenteld, destinatariold, asunto, cuerpo, fecha, leido
Notificacion	id (ObjectId), destinatariold, titulo, mensaje, fecha, leida, tipo (TipoNotificacion)
TipoNotificacion	Enum: ANOTACION, MENSAJE, RECORDATORIO

3.5 Swagger / OpenAPI

Cada microservicio expone su propia especificación OpenAPI (config/OpenApiConfig.java). El API Gateway agrega todas en un único Swagger UI:

Recurso	URL
Swagger UI (unificado)	http://localhost:9090/swagger-ui.html
API Docs Gateway	http://localhost:9090/v3/api-docs
API Docs Académico	http://localhost:9090/academico/v3/api-docs
API Docs Asistencia	http://localhost:9090/asistencia/v3/api-docs
API Docs Comunicaciones	http://localhost:9090/comunicaciones/v3/api-docs

4. Persistencia de Datos

4.1 Estrategia por Servicio

Servicio	Motor	Base de Datos	Propósito
Servicio Académico	MySQL 8.0	db_academico	Estudiantes, profesores, apoderados, asignaturas, evaluaciones, notas, horarios
Servicio Asistencia	MySQL 8.0	db_asistencia	Asistencias (PRESENTE/ATRASO/AUSENTE/JUSTIFICADO) y anotaciones (POSITIVA/NEGATIVA)
Servicio Comunicaciones	MongoDB 6.0	db_comunicaciones	Mensajes y notificaciones (documentos flexibles, sin esquema rígido)

4.2 Spring Data JPA — Servicios Académico y Asistencia

Implementación mediante el patrón Repository de Spring Data JPA:

- Cada entidad tiene su propio Repository que extiende `JpaRepository<Entity, Long>`
- Las relaciones (`@ManyToOne`, `@OneToMany`) son gestionadas por Hibernate con `FetchType.LAZY` por defecto
- El DDL (`CREATE TABLE`) es generado automáticamente por Hibernate (`spring.jpa.hibernate.ddl-auto=update`)
- Se implementa `DataSeeder.java` (`CommandLineRunner`) para poblar datos de prueba al iniciar
- `GlobalExceptionHandler.java` (`@ControllerAdvice`) centraliza el manejo de `EntityNotFoundException` y otros errores

4.3 Spring Data MongoDB — Servicio Comunicaciones

El servicio de comunicaciones utiliza MongoDB para almacenar documentos flexibles:

- Repositorios extienden `MongoRepository<Entity, String>` (ID tipo `String` = `ObjectId`)
- Las colecciones se crean automáticamente al insertar el primer documento
- Los documentos no tienen esquema rígido, permitiendo evolución sin migraciones

4.4 Persistencia con Docker Volumes

Docker Compose define volúmenes persistentes para garantizar que los datos sobrevivan reinicios de contenedores:

- `academico-data` → `/var/lib/mysql` (MySQL servicio académico)
- `asistencia-data` → `/var/lib/mysql` (MySQL servicio asistencia)
- `comunicaciones-data` → `/data/db` (MongoDB servicio comunicaciones)
- `sonarqube-data` / `sonarqube-logs` → Análisis y logs de SonarQube

5. Pruebas Unitarias — JUnit 5

5.1 Herramientas de Testing

Herramienta	Uso
JUnit 5 (Jupiter)	Framework de testing — @Test, @BeforeEach, @DisplayName, @ExtendWith
Mockito	Mocking de dependencias — @Mock, @InjectMocks, @MockBean, when().thenReturn()
MockMvc	Pruebas de controladores HTTP sin levantar servidor real
@WebMvcTest	Test slice para controladores: carga solo el controlador y dependencias mínimas
@DataJpaTest	Test slice para repositorios JPA con H2 en memoria
H2 Database	Base de datos en memoria para tests de integración (no requiere MySQL real)
JaCoCo	Cobertura de código — genera reporte XML en target/site/jacoco/jacoco.xml
SonarQube	Análisis de calidad: cobertura, code smells, vulnerabilidades, duplicaciones

5.2 Tests por Servicio

5.2.1 API Gateway

Archivo	Tipo	Técnica / Cobertura
AuthControllerTest.java	Controlador	MockMvc, Mockito — /auth/login, /auth/cambiar-contrasena
AuthServiceTest.java	Servicio	Mockito, CircuitBreaker mock — login UTP local y login remoto
JwtUtilTest.java	Unitario	Generación, validación, expiración de tokens JWT
UtpControllerTest.java	Controlador	MockMvc — gestión usuarios UTP (/admin/usuarios-utp)

5.2.2 Servicio Académico

Archivo	Tipo	Técnica / Cobertura
ApoderadoControllerTest.java	Controlador	@WebMvcTest, MockMvc — CRUD completo /api/apoderados
ApoderadoServiceImplTest.java	Servicio	Mockito — crear, buscar, actualizar, eliminar apoderado
ServicioAcademicoApplicationTests.java	Integración	H2 en memoria, carga de contexto Spring completo

5.2.3 Servicio Asistencia

Archivo	Tipo	Técnica / Cobertura
AsistenciaControllerTest.java	Controlador	@WebMvcTest, MockMvc — CRUD asistencias
AsistenciaServiceImplTest.java	Servicio	Mockito — operaciones CRUD asistencia

AnotacionServiceImplTest.java	Servicio	Mockito, Feign mock, ArgumentCaptor — verifica publicación RabbitMQ en anotaciones NEGATIVAS
AnotacionEventPublisherTest.java	Messaging	Mock RabbitTemplate — verifica convertAndSend con exchange y routing key correctos
AcademicoFeignClientFallbackTest.java	Fallback	Verifica valores por defecto: null y Collections.emptyList()
RepositoryIntegrationTest.java	Integración	@DataJpaTest, H2 — persistencia de asistencias y anotaciones

5.2.4 Servicio Comunicaciones

Archivo	Tipo	Técnica / Cobertura
MensajeControllerTest.java	Controlador	MockMvc — CRUD mensajes /api/mensajes
MensajeServiceImplTest.java	Servicio	Mockito — operaciones CRUD mensajes MongoDB
AnotacionEventConsumerTest.java	Messaging	Mock NotificacionService — verifica creación de notificacion con apoderado y sin apoderado
RabbitMQConfigTest.java	Config	Verifica beans Exchange (Topic), Queue, Binding correctamente configurados
NotificacionTest.java	Modelo	Verifica construcción y atributos de Notificacion
TipoNotificacionTest.java	Enum	Verifica valores del enum (ANOTACION, MENSAJE, etc.)
MensajeTest.java	Modelo	Verifica construcción y atributos de Mensaje

5.3 Configuración JaCoCo + SonarQube

Plugins Maven configurados en cada pom.xml:

- jacoco-maven-plugin (0.8.11) — prepare-agent (antes de tests) + report (fase verify) → XML en target/site/jacoco/jacoco.xml
- sonar-maven-plugin (3.11.0.3922) — sonar:sonar → envía análisis al servidor SonarQube
- Script automatizado: scripts/sonar-analysis.sh [TOKEN] — ejecuta MVN verify + sonar para los 4 servicios

Cobertura mínima requerida: 60% (indicador 4 de la rúbrica). El proyecto supera este umbral con tests en controladores, servicios, mensajería, modelos, fallbacks y repositorios.

5.4 Resultados SonarQube — Cobertura Real

Servicio	Cobertura	Quality Gate	Líneas de Código
servicio-academico	92.6%	✓ Passed	1.7k
servicio-asistencia	94.1%	✓ Passed	833
servicio-comunicaciones	94.9%	✓ Passed	685
api-gateway	72.7%	✓ Passed	870

☆ colegio-api-gateway Public ✓ Passed Last analysis: 57 seconds ago • 870 Lines of Code • Java, XML E 48 Security A 0 Reliability A 48 Maintainability E 0.0% Hotspots Reviewed 72.7% Coverage 9.6% Duplications
☆ colegio-servicio-academico Public ✓ Passed Last analysis: 8 minutes ago • 1.7k Lines of Code • Java, XML E 16 Security B 7 Reliability A 65 Maintainability E 0.0% Hotspots Reviewed 92.6% Coverage 0.0% Duplications
☆ colegio-servicio-asistencia Public ✓ Passed Last analysis: 6 minutes ago • 833 Lines of Code • Java, XML D 4 Security A 27 Reliability A 59 Maintainability A — Hotspots Reviewed 94.1% Coverage 0.0% Duplications
☆ colegio-servicio-comunicaciones Public ✓ Passed Last analysis: 5 minutes ago • 685 Lines of Code • Java, XML D 3 Security A 23 Reliability A 44 Maintainability A — Hotspots Reviewed 94.9% Coverage 0.0% Duplications

6. Infraestructura — Docker

6.1 Docker Compose (docker-compose.yml)

Servicio	Imagen/Build	Puerto	Dependencias
db-academico	mysql:8.0	—	—
db-asistencia	mysql:8.0	—	—
db-comunicaciones	mongo:6.0	—	—
rabbitmq	rabbitmq:3-managem ent	5672 / 15672	—
servicio-academico	./servicio-academico	8081	db-academico (healthy)
servicio-asistencia	./servicio-asistencia	8082	db-asistencia, rabbitmq, academico
servicio-comunicaciones	./servicio-comunicaci ones	8083	db-comunicaciones, rabbitmq
api-gateway	./api-gateway	9090	academico, asistencia, comunicaciones
frontend	./frontend (Nginx)	3000 → 80	api-gateway
sonarqube	sonarqube:communit y	9000	—

6.2 Healthchecks Configurados

- MySQL: mysqladmin ping -h localhost -uroot -proot (intervalo: 10s, timeout: 5s, reintentos: 10)
- MongoDB: mongosh --eval "db.adminCommand('ping')" (intervalo: 15s, start_period: 30s, reintentos: 10)
- RabbitMQ: rabbitmq-diagnostics ping (intervalo: 10s, timeout: 5s, reintentos: 10)

6.3 Dockerfiles

Archivo	Base Image	Estrategia
api-gateway/Dockerfile	eclipse-temurin:21-jd k-alpine	Multi-stage: build JAR con Maven → ejecuta JAR
servicio-academico/Dockerfile	eclipse-temurin:21-jd k-alpine	Multi-stage: build JAR → ejecuta JAR
servicio-asistencia/Dockerfile	eclipse-temurin:21-jd k-alpine	Multi-stage: build JAR → ejecuta JAR
servicio-comunicaciones/Dockerfile	eclipse-temurin:21-jd k-alpine	Multi-stage: build JAR → ejecuta JAR
frontend/Dockerfile	node:20-alpine → nginx:alpine	Multi-stage: npm run build → copia dist/ a Nginx; proxy /api/ al gateway

7. Patrones de Diseño

7.1 Patrones Implementados

	Patrón	Ubicación	Descripción
✓	API Gateway	api-gateway/	Punto único de entrada: enruta, autentica y autoriza peticiones
✓	Circuit Breaker	api-gateway + servicio-asistencia (Feign)	Resilience4j: umbral 50% fallos, ventana 10 llamadas
✓	Fallback Pattern	AcademicoFeignClientFallback.java	Respuesta por defecto cuando servicio remoto falla
✓	Microservicios	4 servicios independientes	Cada servicio con su propia BD y despliegue independiente
✓	Mensajería Asíncrona	RabbitMQ (Pub/Sub)	Desacopla servicio-asistencia de servicio-comunicaciones
✓	DAO / Repository	*Repository.java (JPA + MongoDB)	Spring Data abstrae el acceso a datos relacionales y documentales
✓	Service Layer (Interface+Impl)	*Service.java + *ServiceImpl.java	Separación entre contrato (interfaz) e implementación
✓	DTO / Eventos	AnotacionEvent.java	Objeto de transferencia para comunicación inter-servicios vía RabbitMQ
✓	JWT (Stateless Auth)	JwtUtil.java	Autenticación sin estado con tokens HMAC-SHA, 8h expiración
✓	Singleton	Beans @Service, @Component, @Bean	Instancia única gestionada por el contenedor de Spring
✓	Global Exception Handler	GlobalExceptionHandler.java	@ControllerAdvice para manejo centralizado de excepciones HTTP

8. Justificación de la Arquitectura de Microservicios

8.1 Por qué Microservicios para el Sistema Escolar

La arquitectura de microservicios fue elegida por las siguientes razones técnicas y de negocio:

- Separación de dominios: Las funcionalidades académicas (notas, evaluaciones), de asistencia (control diario) y comunicaciones (mensajes) tienen ciclos de vida, frecuencias de cambio y equipos de desarrollo distintos.
- Escalabilidad independiente: El servicio de asistencia puede escalar horizontalmente en horas de alta demanda (inicio de jornada) sin afectar los demás servicios.
- Persistencia heterogénea: Los datos académicos requieren integridad relacional (MySQL + JPA), mientras las comunicaciones se benefician de documentos flexibles (MongoDB).
- Resiliencia: El Circuit Breaker garantiza que una falla del servicio académico no bloquee el registro de asistencias (fallback graceful).
- Desacoplamiento asíncrono: RabbitMQ permite que el registro de una anotación negativa dispare notificaciones sin bloquear la respuesta al profesor.

8.2 Cumplimiento de Requerimientos del Cliente

Requerimiento	Solución Implementada
Gestión de usuarios por roles	JWT + Spring Security + ProtectedRoute.jsx (4 roles: UTP, PROFESOR, ALUMNO, APODERADO)
Registro de asistencia y anotaciones	Servicio Asistencia (MySQL) con API REST + OpenFeign hacia Servicio Académico
Notificación automática a apoderados	RabbitMQ Pub/Sub: anotación NEGATIVA → evento → notificación MongoDB
Consulta de notas y calificaciones	Servicio Académico (MySQL/JPA) + páginas MisNotasPage.jsx / NotasList.jsx
Mensajería interna	Servicio Comunicaciones (MongoDB) + MensajesPage.jsx con bandeja de entrada
API documentada para integraciones	Swagger/OpenAPI unificado en http://localhost:9090/swagger-ui.html
Despliegue reproducible	Docker Compose con 10 contenedores, healthchecks y red interna colegio-net

8.3 Recomendaciones de Mejora Futura

- Implementar Eureka Server para Service Discovery dinámico (eliminar URLs fijas)
- Migrar de RestTemplate a WebClient para comunicación no bloqueante y reactiva
- Implementar API Rate Limiting en el Gateway para proteger contra abuso
- Agregar Spring Boot Actuator + Prometheus + Grafana para monitoreo en tiempo real
- Completar tests de integración del Servicio Comunicaciones (requiere MongoDB+RabbitMQ embebidos o Testcontainers)
- Agregar autenticación OAuth2 / OpenID Connect para mayor seguridad enterprise

Resumen de Tecnologías Implementadas

	Tecnología / Patrón	Dónde se usa
✓	React 18 + Vite + TailwindCSS	frontend/
✓	React Router DOM v6	frontend/src/App.jsx
✓	Axios + Interceptores JWT	frontend/src/api/client.js
✓	useState / useEffect	Todas las páginas y componentes
✓	Context API (AuthContext + DarkModeContext)	frontend/src/context/
✓	Spring Cloud Gateway	api-gateway/ (puerto 9090)
✓	Spring Security + JWT (jjwt 0.12.6)	api-gateway/src/main/.../security/
✓	Resilience4j Circuit Breaker	api-gateway + servicio-asistencia
✓	OpenFeign (cliente HTTP declarativo)	servicio-asistencia/src/.../feign/
✓	RestTemplate (comunicación síncrona)	api-gateway/src/.../config/
✓	Spring Data JPA + Hibernate + MySQL	servicio-academico + servicio-asistencia
✓	Spring Data MongoDB	servicio-comunicaciones/
✓	RabbitMQ (Pub/Sub — Topic Exchange)	servicio-asistencia (publisher) + comunicaciones (consumer)
✓	JUnit 5 + Mockito + MockMvc	*/src/test/java/
✓	JaCoCo + SonarQube	*/sonar-project.properties + scripts/sonar-analysis.sh
✓	Docker + Docker Compose (10 contenedores)	docker-compose.yml
✓	Swagger / OpenAPI 3.0 (unificado)	http://localhost:9090/swagger-ui.html